You've already seen that you can apply a general authorization requirement by calling `RequireAuthorization()` on an endpoint or a route group. This is directly equivalent to adding the `[Authorize]` attribute to a Razor Page or MVC controller action. In fact, you can use the same `[Authorize]` attribute on an endpoint if you wish, so the following two endpoint definitions are equivalent:

```
app.MapGet("/", () => "Hello world!").RequireAuthorization();
app.MapGet("/", [Authorize] () => "Hello world!");
```

If you want to require a specific policy (the `"CanCreate"` policy, for example), you can pass the policy name to the `RequireAuthorization()` method the same way you would for the `[Authorize]` attribute:

```
app.MapGet("/", () => "Hello world!").RequireAuthorization("CanCreate");
app.MapGet("/", [Authorize("CanCreate")] () => "Hello world!");
```

Similarly, you can exclude endpoints from authentication requirements using the `AllowAnonymous()` function or `[AllowAnonymous]` attribute:

```
app.MapGet("/", () => "Hello world!").AllowAnonymous();
app.MapGet("/", [AllowAnonymous] () => "Hello world!");
```

This is a good start, but as you saw in chapter 24, you often need to perform resource-based authorization. For example, in the context of the recipe API, users should be allowed to edit or delete only recipes that they created; they can't edit someone else's recipe. That means you need to know details about the resource (the recipe) before determining whether a request is authorized.